

Interim Release Deliverables

Interim Release Demonstration

Our Interim Release Demonstration will present the core Grocery Finder functionality already implemented in our current system. The project includes a working frontend and backend structure, shopping list optimization, aisle-based route generation, and coupon-aware pricing support. Overall, this release demonstrates that Grocery Finder already has a functioning application foundation that supports the main grocery pathfinding workflow promised in our earlier sprint deliverables.

Design Approach

The Grocery Pathfinder application uses a layered, object-oriented architecture split across data, service, and presentation tiers. The data layer has nine SQLAlchemy model classes (User, Store, GroceryItem, ShoppingCart, Coupon, StoreLayout, and others) that handle both state and behavior following standard OOP principles.

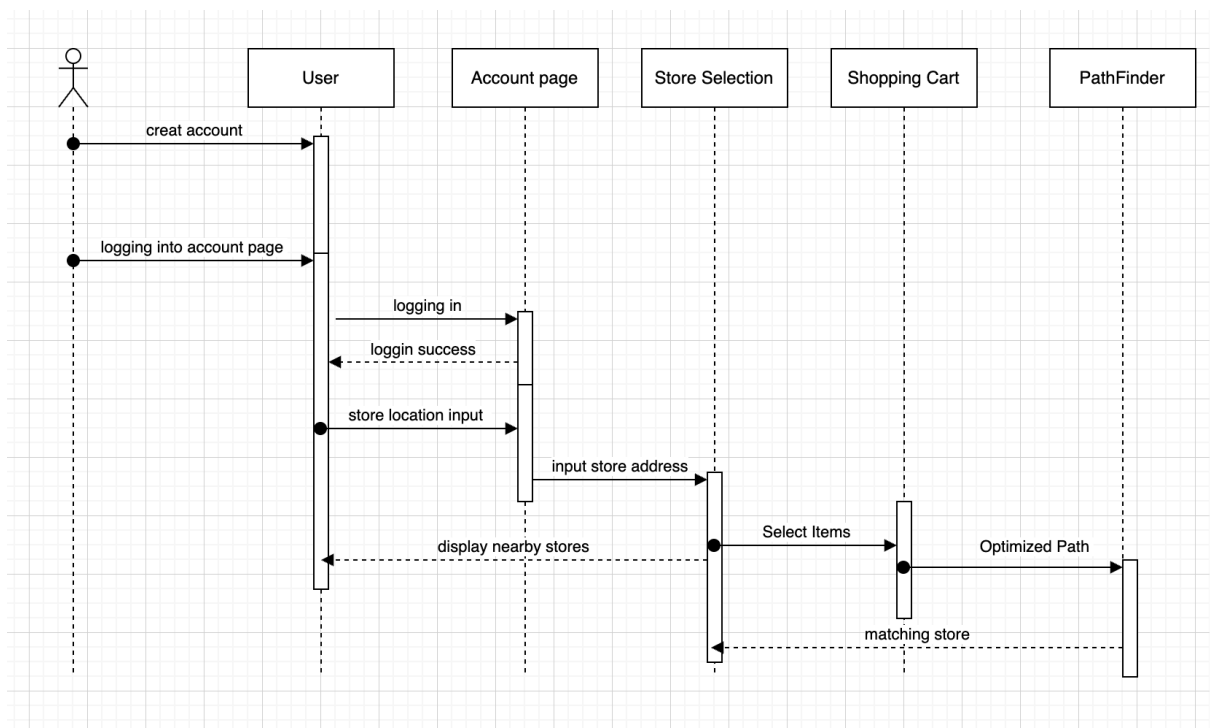
The service layer is where most of the design pattern work lives. It defines ProductMatcher and AislePlanner protocols implemented by several interchangeable algorithm classes: DefaultMatcher, StoreBrandPreferredMatcher (currently not implemented but should be for beta/final release), BudgetAwareMatcher, and CouponAwareMatcher handle product matching, while DefaultAislePlanner and MobilityFriendlyAislePlanner (also currently not fully implemented) cover store routing. Strategy selection runs through abstract factory classes (MatcherFactory, AislePlannerFactory) whose concrete subclasses are chained in a chain of responsibility that reads user context flags (budget mode, store brand preference, coupon mode, accessibility) to pick the right strategy at runtime. CouponAwareMatcher and BudgetAwareMatcher also wrap inner matchers through delegation rather than subclassing,

which keeps the layering clean.

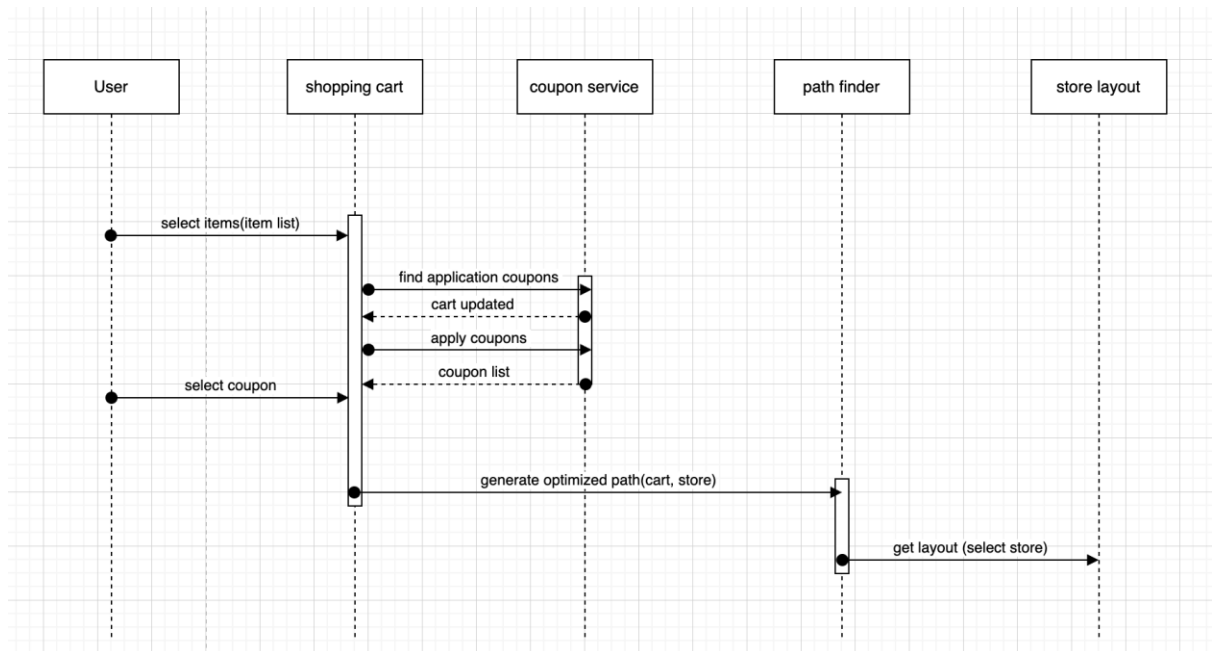
The backend follows MVC: Flask Blueprints as controllers, SQLAlchemy models as the model layer, and JSON API responses as views. The React/TypeScript frontend mirrors this with component-level state management, a centralized API client, and typed interfaces for all data contracts. Other notable pieces include the Haversine formula for a geospatial store proximity, a tiered keyword-matching algorithm for product lookup, and a dual-source coupon system that combines database-linked promotions with hardcoded fallback rules for reliability, since the coupons and inventory are made up.

Sequence Diagrams (Improve)

User Login and Store Selection:



Generate Optimized Shopping Path:



CRC Cards

User	
This class manages user authentication credentials and maintains the user's personal profile data, such as their shopping list. It also processes and stores the user's current location through zipcode input.	ShoppingCart

Cohesion: 7 (Informational)

Coupling: 1 (Data)

Store	
The class maintains the information for individual retail locations, including names, physical addresses, and geospatial coordinates. It also looks at the grocery inventory to identify and searches for available coupons for the user.	Coupon
	ShoppingCart

Cohesion: 7 (Informational)

Coupling:

Coupon: 1 (Data)

ShoppingCart: 1 (Data)

GroceryItem	
This class stores specific attributes for products, such as the item name, unit cost, and current inventory levels. It also tracks the aisle of the grocery item within a store.	ShoppingCart
	Store

Cohesion: 7 (Informational)

Coupling:

ShoppingCart: 1 (Data)

Store: 1 (Data)

ShoppingCart	
This class manages the collection of grocery items the user selects and tracks the quantities of each. It is responsible for calculating the total cost of the order and ensuring this information is saved.	User
	GroceryItem
	Store

Cohesion: 6 (Functional)

Coupling:

User: 2 (Stamp)

GroceryItem: 1 (Data)

Store: 1 (Data)

Coupon	
This class has logic for discounts and matches grocery items to coupons to identify which catalog items are eligible for price reductions.	GroceryItem
	Store

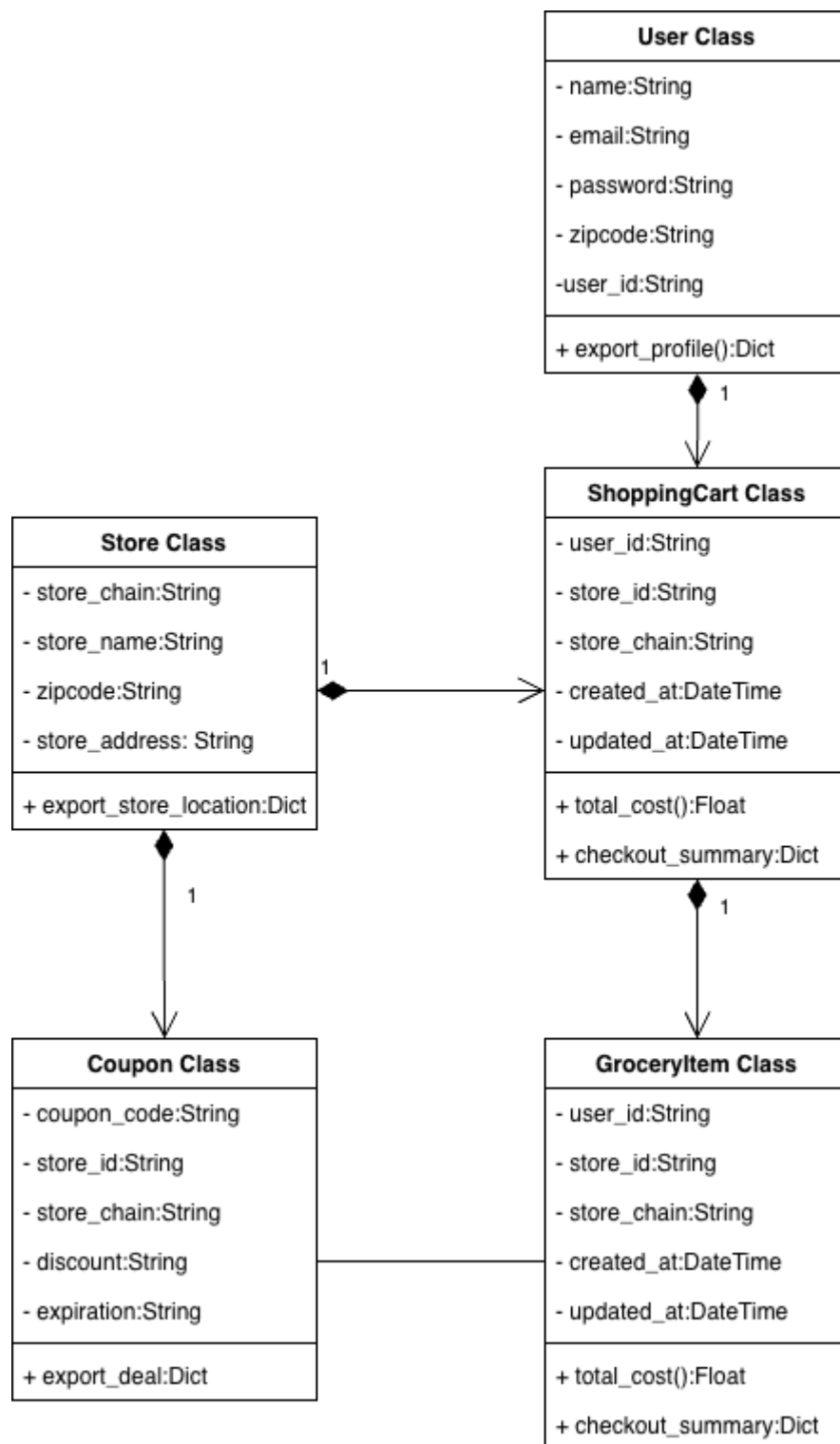
Cohesion: 5 (Communication)

Coupling:

GroceryItem: 3 (Control)

Store: 3 (Control)

Static Class Diagram



Interface Testing Summary

The interface testing session involved five testers, and the overall consensus was that our system was generally intuitive, functional, and easy to navigate. The core tasks of creating an account, logging into the account, selecting a grocery store from a zip code,

building a grocery list, optimizing the shopping path, comparing store prices, and completing the shopping process were completed successfully by all testers with little to no difficulty. Testers found that the login/signup was straightforward, the store selection and search feature efficient, the overall UI user-friendly, and the buttons were labeled clearly. However, there were some issues identified, such as the optimize path button being difficult to access due to how the scrolling of the grocery items was set up. Additionally, there were some inaccuracies in the backend address handling, and it needs to be fine-tuned. Some suggestions included repositioning the optimize path button for easier accessibility and refining the backend address for accuracy. The next steps for the team are to improve backend addressing for improved accuracy and adjust parts of the UI to make some features easier to find and use.

Project Status Report

Currently, the team has begun to pick up steam and has developed the project to create a functional product that meets all of our expected features. From the Sprint 4 release, we have already created a usable interface that allows someone to log in and log out, while saving the login information on the website locally. The website was already capable of searching up stores by zip code and adding items to the cart. For the interim release, our team has further improved upon the previous version of our product. One improvement we made was adding the correct stores and items, since the previous version of the product was using made-up values for the zip code. Now, stores located in the same zip code as Rensselaer Polytechnic Institute can be found through our product. We limited the stores to Hannaford and Walmart. After extensive research, we realized that most APIs that provide real-time accuracy for pulling items from stores require funding that we do not have. The team then consulted with several teacher assistants and student mentors and decided to generate the data ourselves. Using the Kroger API to obtain a list of possible items sold at stores and with a probability distribution for determining the prices of these items, we created synthetic data

for the stores that our product supports. Another improvement from Sprint 4 was the addition of our coupon deals feature, similar to the store items. The team was unable to reliably obtain real-time coupon deals for our product, so the coupon deals had to be synthetically generated again. However, the implemented coupons will act accordingly and deduct some cost from the total price of a shopping cart. Some other issues the team faced were during coding. When clicking on the 'Optimize My List' button, the next page would not display items the user had previously selected, but instead randomly selected items. This error was solved by Sebastian, who discovered that the issues were caused by outdated code that rendered all the data meaningless and fake, resulting in the program randomly guessing which items were in the shopping cart. This error had several negative impacts on development and debugging. One prominent example occurred during the development of the coupon logic, where the developer of that particular feature, Peijing, encountered numerous problems debugging his code because he was mistakenly led to believe it was his code that was the issue. This bug was quite severe, as not catching it could mean failure on showcase day, and has several impacts on other features, such as the 'Optimize My List' feature and the aforementioned coupon feature. Documentation and communication between group members are improving as the team meets all promised deliverables, and overall progress on this project is positive.

Contribution Summary

Sebastian Grammas

My main contributions came from generating the kroger api scraping script to gather the 16000 items of inventory for our database. I also implemented the hardcoded seeding for the coupons and grocery stores in the database. Finally, I dealt with all the backend bugs and issues that arose with the db and the issues that came with adding coupons and other issues within transferring the cart to the optimized path.

Qizheng Sui:

I contributed mainly through project scheduling, deliverable planning, and team coordination for this deliverable. I helped keep the team organized and made sure our work stayed aligned with the requirements from Sprint 1 and Sprint 4. I also supported the preparation of the Interim Release Demonstration by helping define what functionality had been completed and how it should be presented in a clear end-to-end workflow. Compared to the rest of the team, my contribution was strongest in planning, organization, and making sure the deliverable was structured clearly and submitted in a way that matched the course expectations.

Peijing Xu:

My contribution to this project has been working on the development and implementation of the coupon feature on the website. I worked along with Sebastian on ensuring that the coupon arithmetic is solid and bug free. I also contributed to all in-class activities and worked on creating the Class diagrams, CRC cards, and sequence diagrams, and I am also responsible for writing the project status report for this specific document. Finally, I implemented a factory design pattern in the backend of our project in accordance with our class's Best Practices.

Alan Lee:

My contributions to this project have been updating the Jira board with tasks to be completed and checking in with the group on their progress, working on the Makefile for our project so that it could easily be compiled and run, and ensuring that dependencies are installed before being run. I also contributed to all in-class activities and worked on creating a summary of all the feedback from the interface testing done during class, and wrote down changes that should be made. I made improvements to the CRC cards and Class Diagram

made during class. I expanded on how the coupling and cohesion scores were determined and expanded the class diagram to fit our source code more.